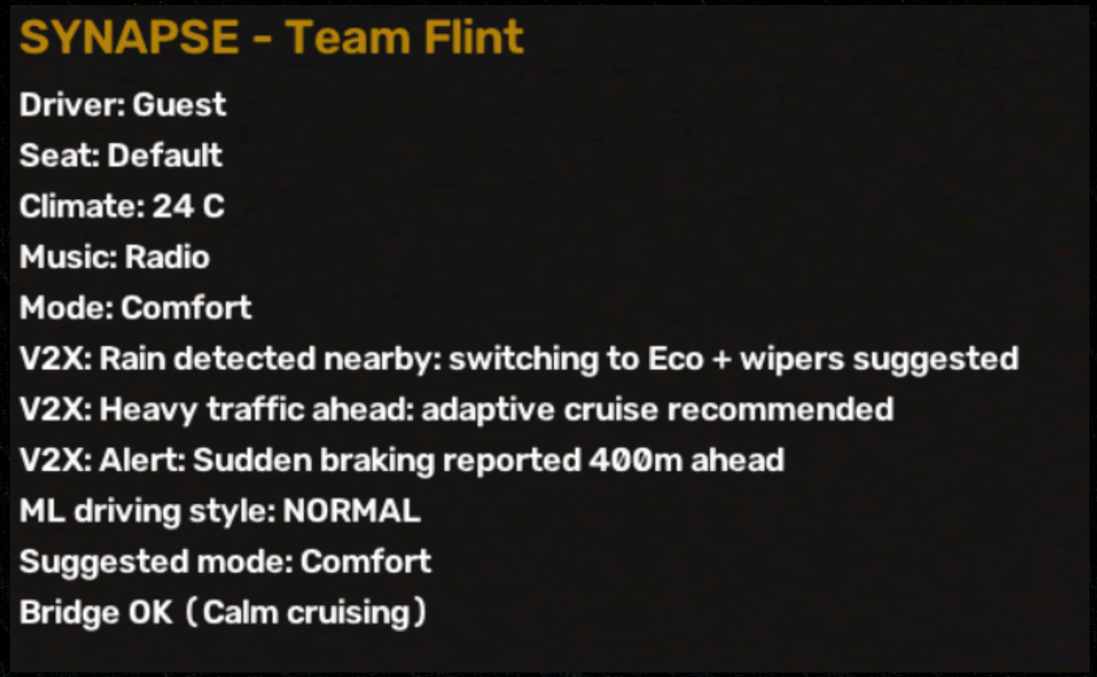
SYNAPSE

Edge AI for Personalized and Connected Vehicles

Team FLINT
B.Vignathi
vignathi123@gmail.com

Introduction

Every driver is unique, and every drive is distinct.
Synapse is my effort to create a car that truly understands both.
It recognizes who is behind the wheel and adjusts to their driving style in real time.
All of this happens on the device itself, with no cloud connection needed.



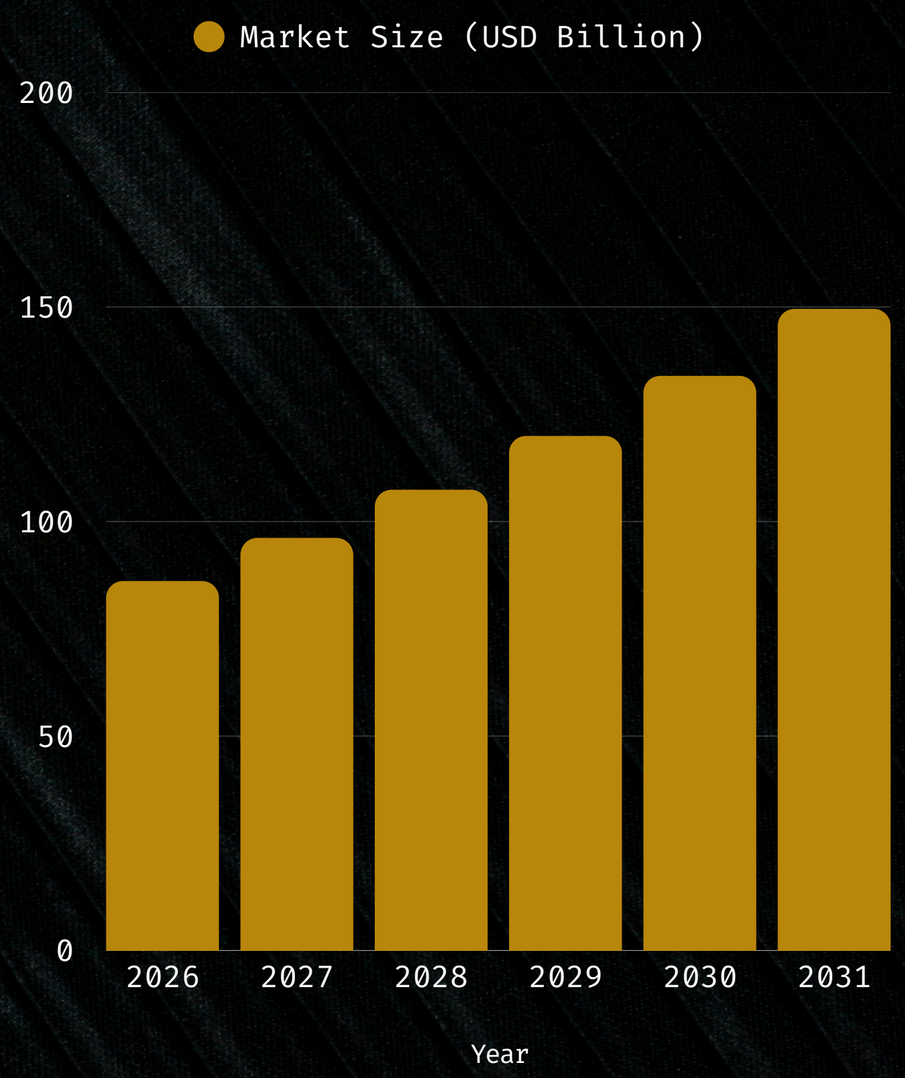
Problem Statement

Edge AI for Personalized and Connected Vehicles

- The personalization stops when it is not connected — no storage of seat position, climate control, or music preferences without a network connection, and no storing of driving data beyond your control
- Incompatible automotive data formats: onboard sensors, insurer portals, fleet management systems, and carmakers' clouds do not share a common language, thus preventing predictive maintenance and usage based insurance
- Personalization and connectivity are viewed as two separate issues — while cars require both at once

Market Size and Future Potential

- Market value of connected vehicles: $86.13 billion (2026) to $149.56 billion by 2031 at a 11.67% CAGR (Mordor Intelligence)
- Drivers behind growth: Monetization of the OTA feature, usage-based insurance, and software defined vehicles becoming an industry trend
- Importance for Synapse: Personalization and interoperability are directly involved with all three of these drivers — hence the tangible path to scalability for the solution



*2027-2030 values are estimated from published 2026/2031 figures using stated CAGR

Objective and Approach

Objective: Deliver a functional implementation, not just an idea – one that achieves true cabin personalization and addresses data interoperability issues for the car.

Approach: Two components that work together

- Edge AI Component: On-device driver recognition & driving style prediction
- Interoperability component: Translates data from sensors on vehicles to telematics format in the cloud
- Fully developed and deployed – not just concepts

Solution by Synapse: On-device cabin personalization using edge AI and an interoperability bridge between different vehicle data formats.

Solution Overview: Personalization

Component 1: On-device driver identification

Local face detection recognizes the driver — all information stays within the vehicle.
Automatic loading of personal settings: seat configuration, cabin temperature, preferred music genre, drive mode.

Component 2: Real-time driving style prediction

The trained machine learning algorithm classifies driving style — slow, normal, or aggressive, based on live accelerometer + gyroscope readings.
Four different algorithms were evaluated and compared, with the best performing selected (I have listed the details on Results slide.)

Component 3: True edge deployment

The model was exported to ONNX format and verified in 2.84ms inference time
Ready for real-world embedded devices (Raspberry Pi / Jetson Nano), not only as a proof-of-concept on a laptop.

Solution Overview: Connectivity+Novelty

API Bridge (Interoperability)

Converts sensor data to telematics in a common cloud format

Addresses an existing automotive issue:

Onboard sensors vs. cloud systems use different data formats.

Connected Vehicle Environment (V2X)

Simulated data stream: traffic volume, weather, nearby incident notifications

Advice uses both personal parameters and external factors – such as rain and traffic that may override "Sport mode"

What makes this novel?

- Existing approaches address personalization and connectivity separately Synapse combines both into one pipeline
- Synapse is backed by real comparative model testing, not a single unverified choice
- An API interoperability which shows real working code!

Challenges Faced

- Noisy single-instant sensor data:

Raw readings gave only 44% prediction accuracy. Solved using feature engineering (motion magnitude + rolling-window statistics), reaching 90.3%

- Vision-only face recognition limits:

Glasses/hats can affect accuracy, a known real-world limitation. Production systems would pair this with additional signals like key-fob or Bluetooth pairing.

- No public dataset for personal preferences:

This data is private and shouldn't exist publicly. Solved with a transparent, rule-based preference-mapping layer instead of fabricating a dataset.

- Ensuring genuine edge-readiness:

Verified by exporting to ONNX and measuring real inference latency, instead of assuming deployability.

Technical Implementation

- Language: Python
- On-device face recognition: OpenCV (LBPH)
- ML models trained & compared: Logistic Regression, SVM, Gradient Boosting, Random Forest
- Edge deployment: ONNX + ONNX Runtime
- Interoperability bridge: FastAPI + Uvicorn
- Data storage: JSON-based driver profiles and connectivity context
- Training data: real public driving-behavior dataset (accelerometer + gyroscope)

Results and Achievements

Model comparison results:

model	accuracy	f1_macro	train_time_s	avg_inference_ms
Random Forest	0.902606	0.902153	1.570097	0.039263
Gradient Boosting	0.784636	0.784459	3.860308	0.002996
SVM (RBF kernel)	0.552812	0.525623	0.163414	0.123460
Logistic Regression	0.541838	0.530379	0.061636	0.001727

- Random Forest was selected after comparing it against three other algorithms (Logistic Regression, SVM, Gradient Boosting) on identical data. It achieved the highest accuracy (90.3%) and F1-score (0.902), significantly outperforming the linear models by capturing non-linear patterns in motion-sensor data, while also training faster than Gradient Boosting, making it the most practical choice for a real-time, on-device system.
- Feature engineering alone improved accuracy from 44% → 90.3%
- ONNX-converted model: 2.84ms inference latency real-time.
- Fully working end-to-end pipeline demonstrated live

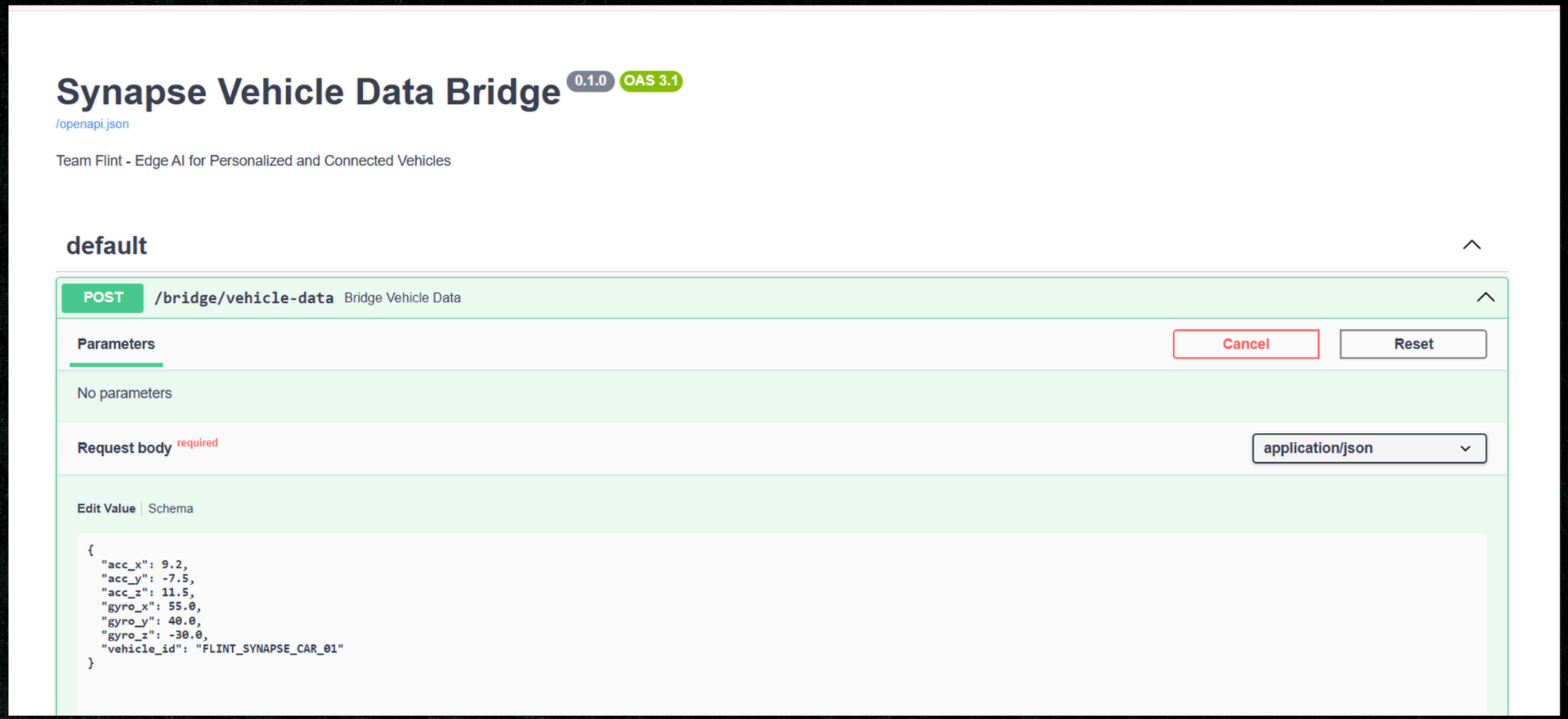
Demonstration

- Full recorded video demonstration included in the submitted Google Drive folder

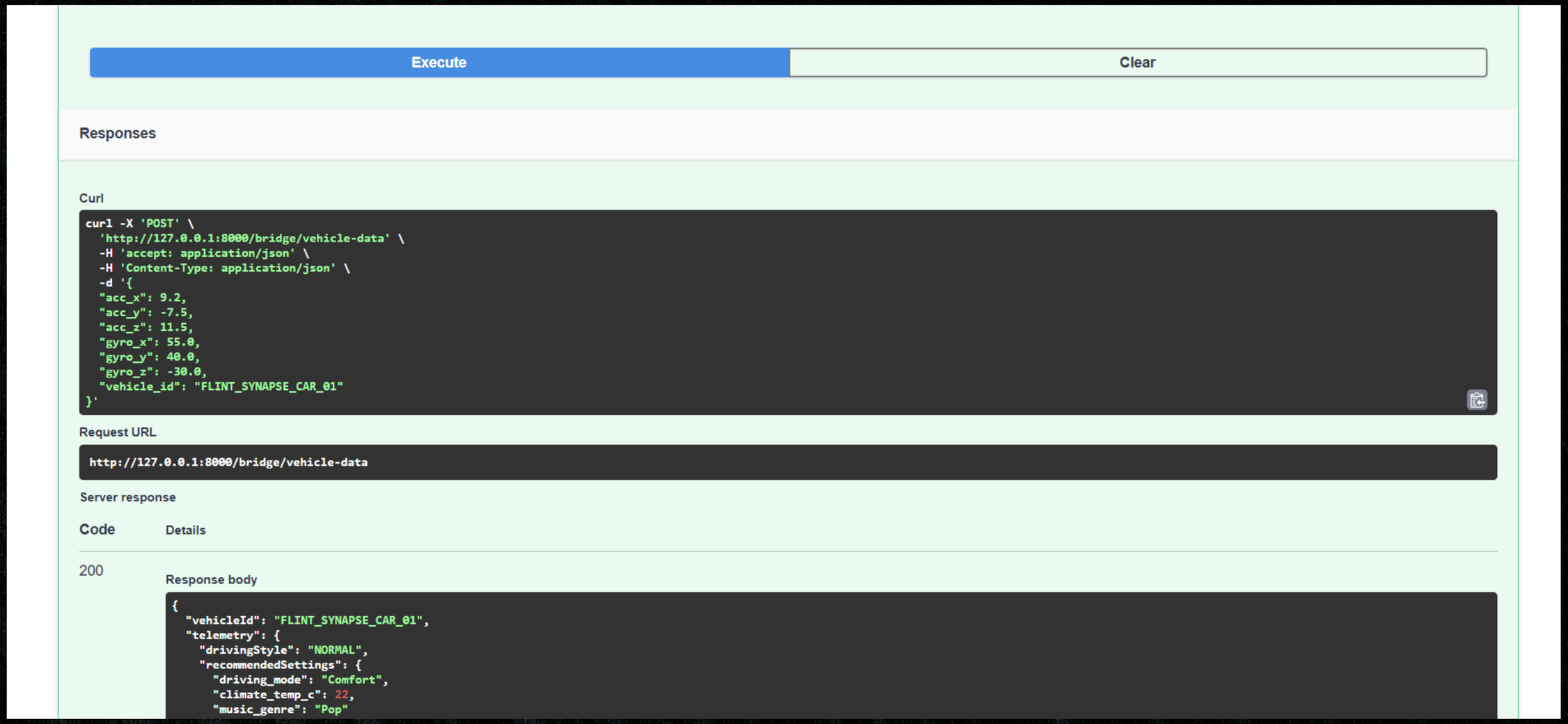
- Video shows:
  Driver recognition switching between personalized profiles
  Real-time ML-driven driving-style prediction
  API bridge translating sensor data into cloud format live
  V2X-based recommendation changes

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\vijay> cd "C:\Users\vijay\OneDrive\Desktop\SYNAPSE_TATA"
PS C:\Users\vijay\OneDrive\Desktop\SYNAPSE_TATA> python api_bridge.py
INFO:     Started server process [32348]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning:
X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
INFO:     127.0.0.1:55509 - "POST /bridge/vehicle-data HTTP/1.1" 200 OK
C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning:
X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
INFO:     127.0.0.1:55511 - "POST /bridge/vehicle-data HTTP/1.1" 200 OK
C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning:
X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
INFO:     127.0.0.1:55512 - "POST /bridge/vehicle-data HTTP/1.1" 200 OK
C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning:
X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

API bridge running locally, successfully processing real requests (200 OK)



Sending a live sensor payload to the Synapse API bridge



Bridge response — raw sensor data translated into standardized cloud telematics format

Future Enhancements

- Real V2X/MQTT integration with live connected-vehicle infrastructure
- Deployment and testing on physical edge hardware (Raspberry Pi / Jetson Nano)
- Multi-signal driver identification (face recognition + Bluetooth pairing) for robustness
- Federated learning across vehicle fleets: collective improvement without centralizing private driver data
- Expanded behavioral dataset for richer personalization signals

Concluding,
Synapse proves personalization and connectivity can work together, on-device, without cloud dependency. Backed by real, tested results: 90.3% model accuracy, 2.84ms edge inference, a working interoperability bridge

Project Plan

Phase	My Plan
Now — Stage 1	Working virtual PoC, validated model, live end-to-end demo
Stage 2 (if shortlisted)	Refine model, robustness testing, present to Tata Technologies' SME jury
Stage 3 (if shortlisted, 2-3 months)	Deploy to real edge hardware, integrate genuine V2X/telematics APIs, build physical/functional prototype

Each stage builds toward a production-ready edge AI system, a validated, deployable framework for personalized and connected vehicle intelligence.

Thank You!